

CHAPTER 9

**HACKING
HARDWARE**

This book discusses at length logical threats to software across all levels, from application to system to network. But what about threats to the hardware and the physical protection mechanisms that safeguard the information assets they carry? This chapter reviews attacks on mechanisms that protect the devices themselves and provides an introduction to reverse engineering hardware devices to probe even deeper into the information they store.

Well-connected embedded devices are incredibly prevalent, whether it's the ubiquitous mobile phone to the ever-popular iPad. From home to work to the coffee shop, a user may utilize the same device to access multiple networks via different mediums, including GSM, Wi-Fi, Bluetooth, and RFID. These devices present a significant risk to organizations as handhelds grow in complexity and become pervasive in the enterprise and home.

Physical access controls and endpoint device security are often encountered by attackers well before they ever get to a network access point or a login prompt. Understanding how attackers bypass these security mechanisms is the key to helping secure infrastructure protection mechanisms.

This chapter presents examples of tools and techniques commonly used to bypass physical and hardware security. We begin with a discussion of bypassing physical door locks, move through cloning of physical proximity access cards and then into attacking hardware devices including password-protected hard disks and the Universal Serial Bus (USB), and conclude with a brief introduction of tools and techniques for reverse engineering devices to illustrate some of the fundamental principles of hardware hacking.

PHYSICAL ACCESS: GETTING IN THE DOOR

Obviously, attacking hardware devices requires physical access to the device. Here we've included a discussion of common techniques to bypass perhaps the most common physical access control mechanism utilized today: the locked door.

Lock Bumping

One of the oldest forms of physical security is the lock. Locks have traditionally been used to secure doors, racks, cases, and just about everything else used to protect computing infrastructure. Locks secure an apparatus by using a series of pins that restrict the mechanism from turning. In standard locks, there are two sets of pins: the driver pins and the key pins. The *driver pins* are suspended by springs and push down on the *key pins*. When inserted into the lock, the key pushes the key pins against the driver pins to align a clear path for the mechanism. Once the pins have been aligned, the mechanism is clear and allows the lock to be turned. The user turns the key and the lock opens. Figure 9-1 illustrates a standard lock in cross-section, showing how the pins are aligned by the inserted key.

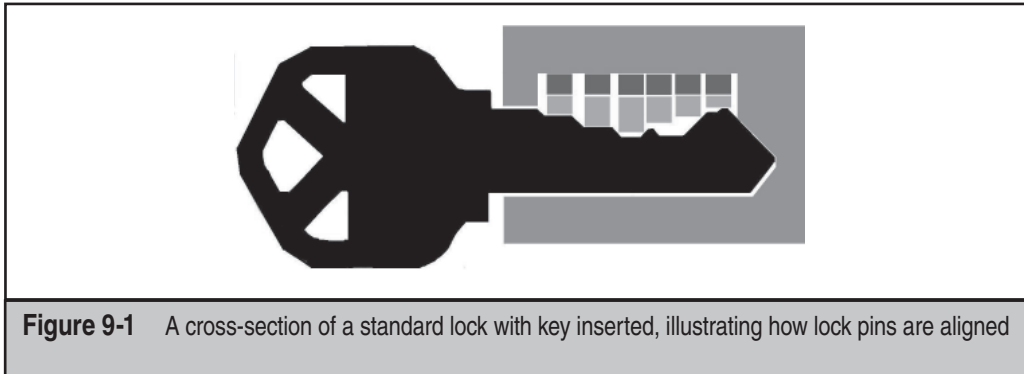


Figure 9-1 A cross-section of a standard lock with key inserted, illustrating how lock pins are aligned

Lock bumping (en.wikipedia.org/wiki/Lock_bumping) allows an attacker to use a single key to open nearly any lock of the same type. Lock bumping works by taking advantage of Newtonian physics. The method is very simple. A standard key pushes the pins into the correct alignment and then the user turns the key. A specially constructed key called a *bump key* has teeth that sit below the key pins. When a bump key is inserted into any standard lock, and then struck (or “bumped”), each of the tips on the bump key transfers the force to the key pins causing them to “bump” into place temporarily for just a fraction of a second. This window of alignment is enough to allow the lock to turn (with some good timing and practice!). Special tools have been developed to assist bumping locks, but a standard screwdriver or anything that can give a gentle but firm strike to the bump key will suffice. Figure 9-2 shows a standard key compared to a bump key, illustrating the short, even-height teeth on the bump key that are designed to impart the necessary force to align the pins in any standard lock. Bumped locks seldom leave evidence of tampering and a practiced individual can bump a lock faster than someone with the real key can open it!

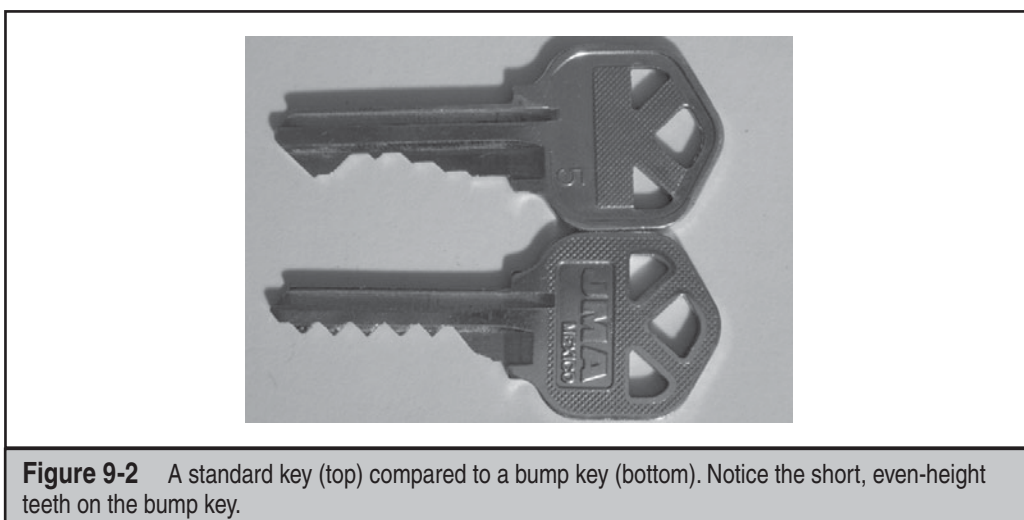


Figure 9-2 A standard key (top) compared to a bump key (bottom). Notice the short, even-height teeth on the bump key.

CAUTION

Repeated bumping can damage or destroy a lock! Use bump keys only on practice locks and locks you are authorized to test. It may be illegal to possess or carry bump keys in your locality.

Bump Key Countermeasures

Few locks are designed with mitigations to bump keys. To make matters worse, two bump keys will open nearly 70 percent of the locks used to protect doors in North America.

There are a few providers of locks that have been known to be bump key and lock pick resistant. Medeco (medeco.com) and Assa Abloy (assaabloy.com/en/com) are two of the more well-known brands. Use their locks on critical assets and to protect important areas.

Medeco locks add an additional layer of security by employing a *sidebar*. The sidebar is an addition pin that must be aligned before the lock can turn. The sidebar aligns only after all of the pins have been aligned and then turned to the correct angle. This additional countermeasure makes both picking and bumping Medeco locks difficult. However, recent research has shown that older Medeco sidebar-type locks can be picked or bumped (see thesidebar.org/insecurity/?p=96).

For critical assets, do not rely on locks alone. The common, compensating physical controls, including using multiple lock devices (for example, a keypad or fingerprint reader in addition to standard lock), video monitoring, guards, and intrusion alarms are also recommended to mitigate the risk from bypassing physical locks.

TIP

Cable locks commonly used to secure laptop computers are even more vulnerable, as a hacker once demonstrated with a Kensington lock being cracked in less than two minutes using a plastic pen barrel and a toilet paper tube.

Cloning Access Cards

Many secure facilities require that an access card be used for entry in addition to other security measures. These cards normally come in one of two types: magnetic stripe (*magstripe*) or RFID (Radio Frequency Identification; these are often referred to as *proximity cards*). In this section, we discuss how to create a clone of each type of card and then replace key information on the cloned card with custom data that can be used to gain physical access.

Hacking Magstripe Cards

Most magstripe cards conform to ISO standards 7810, 7811, and 7813, which define a standard size and specify that the card contains three tracks of data commonly referred to as tracks 1, 2, and 3. The majority of magstripe cards contain no security measures to protect the data stored on the card and encode the data on the card in the clear. As a result, magstripe cards are trivial to clone and reuse.

Tools are available from several providers to clone, alter, and update magstripe card data. The reader/writer pictured in Figure 9-3 is available from makinterface.de, and it

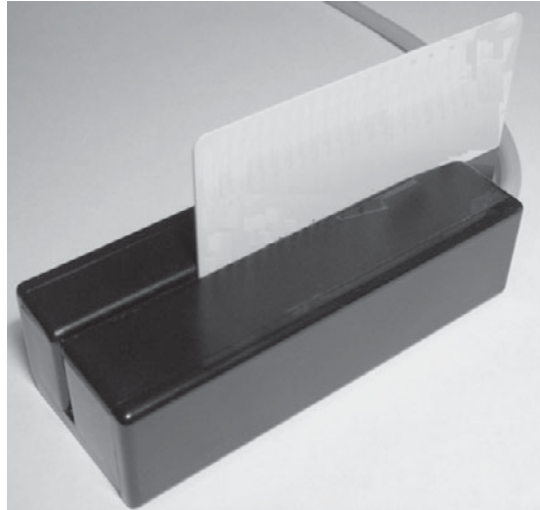


Figure 9-3 A magstripe card reader/writer

comes with the Magnetic-Stripe Card Explorer software shown in Figure 9-4. This tool allows anyone to read, write, and clone access cards. Many cards contain custom data that can be altered to nefarious ends.

Cloning, altering, and writing magstripe cards is a fairly simple process once the data has been acquired from the source card. Figure 9-4 shows Magnetic-Stripe Card Explorer software displaying card data in Char, Binary, or ISO formats.

The data displayed by the Explorer can contain a wealth of information: ID number, serial number, social security number, name, address, and account balances are all common information stored on magstripe cards. This data is often in a custom format and needs to be decoded to human-readable form.

Many times doing a quick analysis of the data is enough to predict how to create a cloned card. Many access cards simply contain an ID or other sequential number. Brute-forcing card values can be a quick way to gain access to a system or bypass a panel. The simplest way to analyze the card data on the three tracks is to read multiple cards of the same type. Once the data has been acquired, use a diff tool to do a visual inspection of the data. If you can correlate what context the data is used in, then decoding it becomes trivial. For example, following is the data from two different cards—notice that only a few bits differ between the two track data rips (in bold).

```
Card 1: Track 1: 001000000111100010010101011000111110011000001001
Card 2: Track 2: 001000000111100010010101100000111110011000001001
```

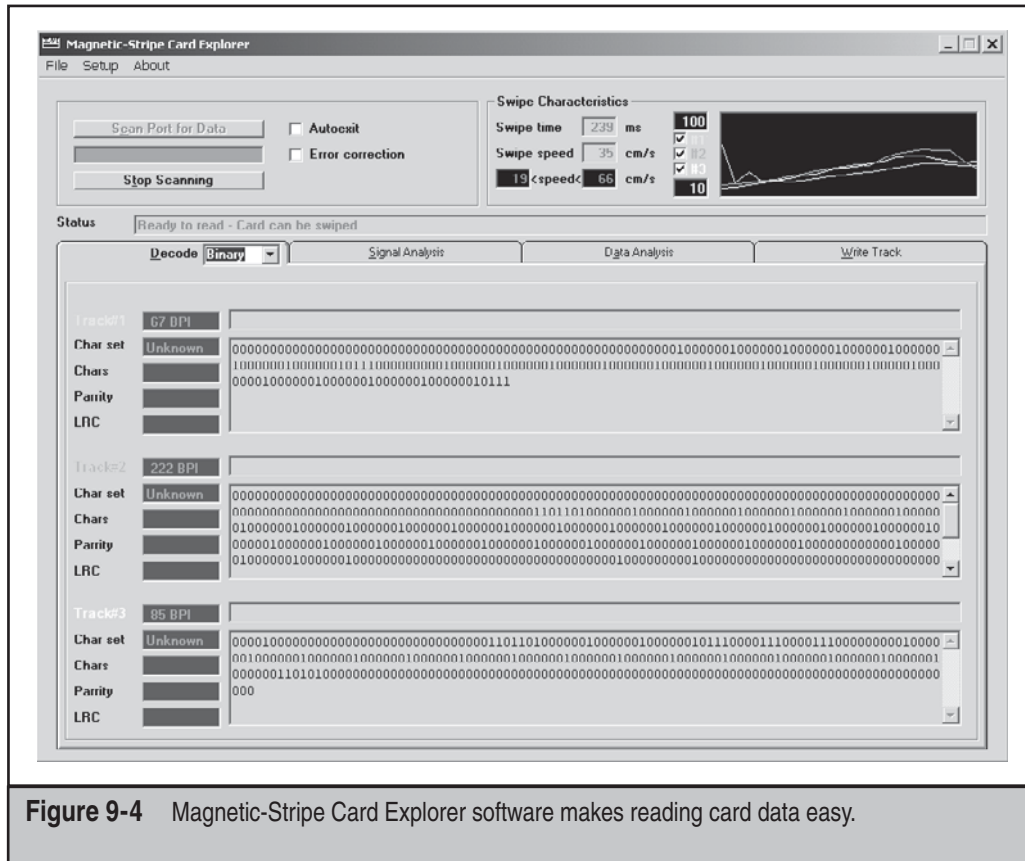


Figure 9-4 Magnetic-Stripe Card Explorer software makes reading card data easy.

These bits likely represent different card IDs. In the prior example, we can see the two different cards are sequential and predict what the next or previous card's value might be based on this pattern.

Writing data back to a card is as simple as choosing which track you want to write the data to. The only tricky part is that many tracks include checksum data to verify that the data on the card is valid or the card wasn't damaged. If there is a checksum, you'll have to determine what checksum is being used and then recalculate a new one before the card can be used. Sometimes a card contains a checksum but they aren't actually used by the reader. Figure 9-5 shows Magnetic-Stripe Card Explorer writing custom data to a card.

CAUTION

Writing data back to a magnetic stripe card can potentially corrupt the source card, causing the card to be rejected or to malfunction during use. Use only disposable cards for testing or reading.

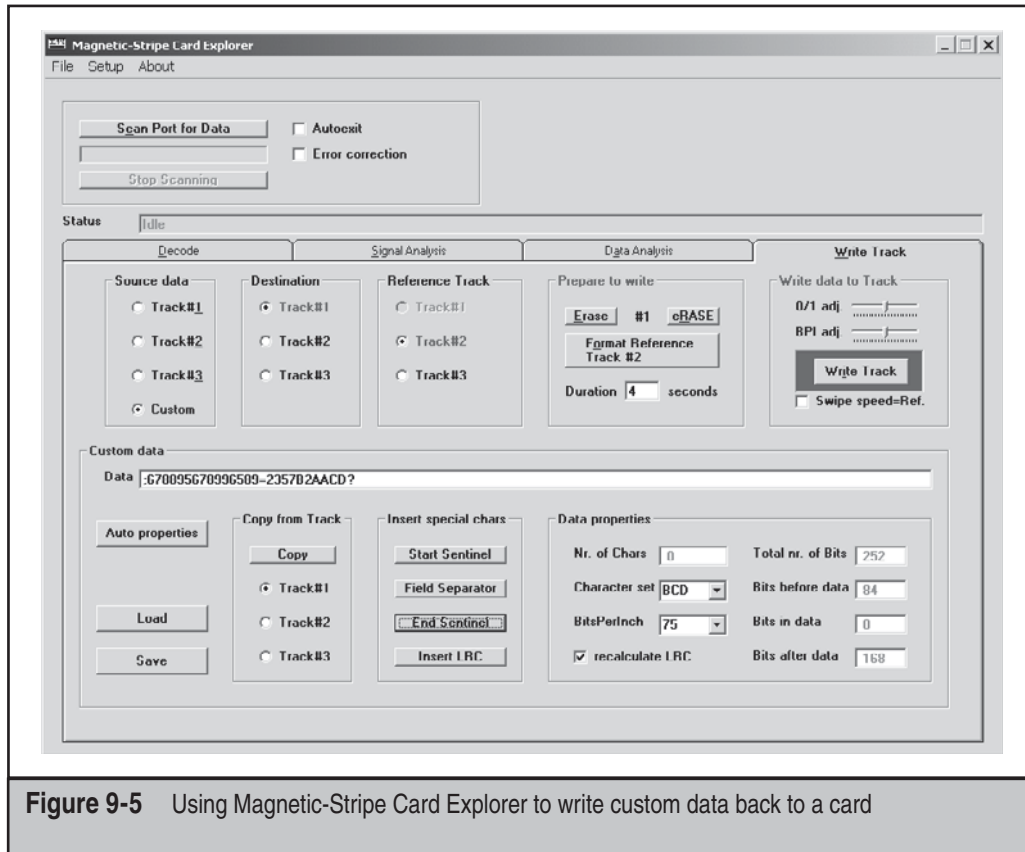


Figure 9-5 Using Magnetic-Stripe Card Explorer to write custom data back to a card

Hacking RFID Cards Magstripe systems are being deprecated in favor of RFID card systems (see en.wikipedia.org/wiki/RFID for more background). RFID is commonly used to provide access to facilities and is also starting to be used in payment systems around the world. Most card access RFID systems operate on one of two different spectrums: 135 kHz or 13.56 MHz. Just like magnetic stripe cards, many RFID cards are unprotected and can be as easily cloned for reuse for entry into systems. More and more RFID cards are starting to employ custom cryptography and other security measures to help mitigate these risks.

The most common RFID card in usage is from HID Corp, which uses a proprietary protocol. Initial research to clone HID cards was performed by Chris Paget in 2007, but this research was never published widely after HID sent a letter to Paget's employer accusing him of possible patent infringement over some materials used in the research.

Hardware tools are available to both read from and imitate common RFID cards, however. Preassembled devices and kits are available from openpcd.org/ for the reader, and the clone device is available at openpcd.org/openpicc.0.html.

A more advanced version of an RFID reader/writer is the proxmark3 device. The proxmark3 has an on-board FPGA built in to allow for the decoding of different RFID protocols. This tool isn't for the faint of heart, or short of budget, as it requires the parts and circuit board to be custom assembled by the user and is no longer supported by the maker. For more information, see the proxmark3 at cq.cx/proxmark3.pl.

A third option for intercepting and decoding RFID traffic is the Universal Software Radio Peripheral (USRP). The USRP can intercept the raw radio waves that then have to be decoded by the user, so this also is a more advanced tool. A properly populated USRP can send and receive raw signals on the common RFID frequencies, allowing it to intercept and imitate cards. A fully configured USRP costs around \$1,000 and the decoding software has to be written per protocol.

Countermeasures for Cloning Access Cards

When it comes to mitigating cloning attacks like the ones just covered, we are unfortunately at the mercy of the access card vendors in most cases. Many vendors' initial goals were to make the access technology as inexpensive as possible, thus proper security and cryptography are not accounted for. Now, due to the widely deployed infrastructure of existing access systems, there is substantial inertia on the part of these vendors to change the features of their systems to resist these types of attacks. As researchers expose more weaknesses (for example, the Mifare card system attack; see en.wikipedia.org/wiki/MIFARE#Security_of_MIFARE_Classic), additional pressure is mounting on vendors to supply a secure solution.

Many newer RFID access systems implement a full cryptographic challenge-response algorithm to help prevent cloning, replay, and other attacks. When the card is energized by the reader, a challenge is sent to the RFID card, which is encrypted and signed by the private key stored on the card and sent back to the reader. The reader validates the response before allowing the holder of the card to access the protected resource. Even if the entire conversation is intercepted, the attacker cannot use the same response twice. Some of these systems implement widely accepted cryptographic algorithms, whereas others implement proprietary encryption that should raise significant concerns among buyers ("don't roll your own crypto" is one of the long-accepted principles of secure design). As RFID systems become more commonplace, more robust countermeasures like challenge-response protocols and strong encryption may become increasingly prevalent—or at least we hope they will!

CAUTION

It should be noted that the tried-and-true method of tailgating someone with valid credentials continues to be the most effective way into many secure areas.

HACKING DEVICES

Assuming an attacker has successfully bypassed any lock-based controls at this point, attention now turns to the devices that store sensitive information. We've included some examples of device hacking in this section to illustrate approaches to bypassing common device security features.



Bypassing ATA Password Security

ATA security is a common safeguard used by companies to deter the usage of a stolen laptop. The ATA security mechanism requires that the user type a password before a hard disk can be accessed by the BIOS. This security feature does not encrypt or protect the contents of the drive, only access to the drive. As a result, it provides minimal security. Many bypass products and services exist for specific drives; however, the most common and easiest to perform is simply to hot-swap the drive into a system with ATA security disabled.

Many drives accept the ATA bus command to update the drive password without having first received the password. This is the result of a disconnect between the BIOS and the drive. Many ATA drives assume the BIOS has authenticated the ATA password before, allowing the user to send a `SECURITY SET PASSWORD` command to the ATA bus. If the BIOS can be fooled into just sending the `SECURITY SET PASSWORD` command, the drive will simply accept it. Figure 9-6 shows two ATA disk drives being prepared for password unlock.



Figure 9-6 Two ATA disk drives ready to have their passwords bypassed

The hot-swap attack works as follows. Find a computer that is capable of setting ATA passwords and an unlocked drive. Boot the computer with the unlocked drive and enter the BIOS interface. Navigate to the BIOS menu that allows you to set a BIOS password, as shown in Figure 9-7. Carefully remove the unlocked drive from the computer and insert the locked drive.

CAUTION

Shorting the leads on the hard drive typically causes the computer to reboot and possibly damages the logic board.

Once the locked drive has been inserted into the computer, set the hard-disk password using the BIOS interface. The drive will accept the new password. Reboot the computer, and when the BIOS prompts you to unlock the drive, the new password should work, bypassing the old one set by the prior user. The password can be cleared from the system if a new password is not desired.

CAUTION

Hot swapping ATA drives may potentially damage the drive, the drive's file system, the computer, or yourself. Take precaution and use this technique at your own risk.



ATA Hacking Countermeasures

The best defense against ATA drive password bypass is to avoid it: do not rely on ATA security to protect drives from tampering or to protect the contents of the drive. Many ATA drives are trivial to bypass, and password protecting them provides a false sense of security. As an alternative to ATA password security, use full disk encryption to protect the entire contents of the drive or sensitive partitions on the drive. Three common products that provide disk encryption are BitLocker (en.wikipedia.org/wiki/BitLocker_Drive_Encryption), TrueCrypt (truecrypt.org), or SecurStar (securstar.com).

```
***** System Security *****
Primary Password: Disabled
Admin Password: Disabled

*** Hard-disk drive password(s) ***
System Primary: Disabled
```

Figure 9-7 A BIOS menu for configuring ATA disk drive passwords

NOTE

See Chapter 4 for a discussion of the “cold boot” attack that can bypass certain disk encryption implementations.

**USB U3 Hack**

One of the easiest ways into a system is by using a USB flash drive that implements the U3 standard. The *U3 system* is a secondary partition included with USB flash drives made by SanDisk and Memorex, like those shown in Figure 9-8. The U3 partition is stored on the device as read only, and it often contains free software for users to try or download. The U3 partition menu is configured to execute automatically when the USB stick is inserted into certain computers.

The U3 hack works by taking advantage of the autorun feature built into Windows. When inserted into a computer, the USB flash drive is enumerated, and two separate devices are mounted: the U3 partition and the regular flash storage device. The U3 partition immediately runs whatever program is configured in the autorun.ini file on the partition. Each manufacturer provides a tool to replace the U3 partition with a custom ISO file for branding or deleting the partition. The partition can be overwritten using the manufacturer’s tool to include a malicious program that executes in the context of the currently logged-on user. The most obvious attacks are to read the password hashes from the local Windows password file or install a Trojan for remote access. The password file can be e-mailed to the attacker or stored on the flash drive for offline cracking later using tools like fgdump (see Chapter 4).



Figure 9-8 USB drives that implement the U3 standard

A USB flash drive–based tool like this can be built in a few easy steps. First, create a custom autorun script to launch a command script when you insert the USB device into the computer, as shown in the following example autorun.inf file:

```
[autorun]
open= go.cmd
icon=autorun.ico
```

Next, create a script to run programs, install tools, or perform other actions, as in the following example, when we call go.cmd:

```
@echo off
if not exist \LOG\%computername% md \WIP\%computername% >nul
cd \WIP\CMD\ >nul
.\fgdump.exe
```

Once you’ve assembled the script and utilities, copy the files to the U3CUSTOM folder provided by the U3 device manufacturer or use a tool like Universal_Customizer (hak5.org/packages/files/Universal_Customizer.zip). The ISOCreate.cmd included with Universal_Customizer can package up the autorun program, executables, and scripts in the U3CUSTOM directory into an ISO to be written to the U3 device.

The final step is to write the ISO to the flash disk with the Universal_Customizer.exe, as shown in Figure 9-9.

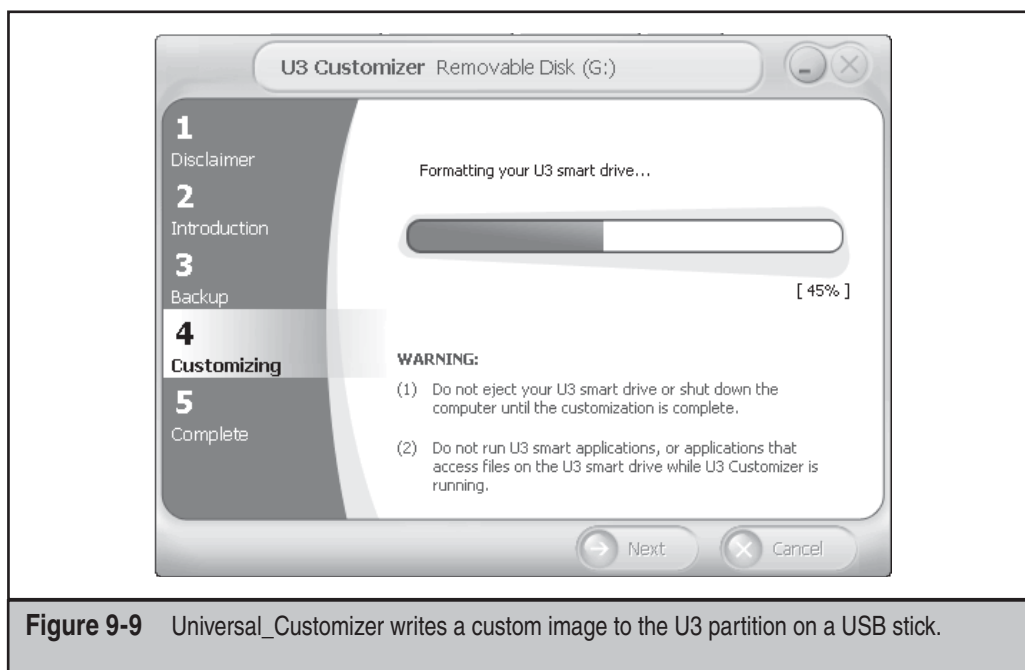


Figure 9-9 Universal_Customizer writes a custom image to the U3 partition on a USB stick.

The U3 stick is now armed and ready for use. Any computer that has autorun enabled will launch the `fgdump.exe` program and record the password hashes. Additional information on creating U3 scripts and several premade U3 packages can be found at wiki.hak5.org/index.php?title=Switchblade_Packages.

CAUTION

The U3 device will not differentiate between computers and will infect or compromise any computer it is inserted into. Be careful not to infect yourself.



U3 Hack Countermeasures

This attack works because of the autorun feature in Windows and other operating systems. The attack can be counteracted in one of two ways. One way is to disable autorun on the system as discussed at support.microsoft.com/kb/953252. Another approach is to hold down the `SHIFT` key before inserting a USB stick on a per-use basis; this prevents autorun from launching the default program.

Even with autorun disabled, it's important to note that a malicious device may still infect files or programs using other mechanisms than the one discussed. When in doubt, never insert an untrusted device into your computer!

DEFAULT CONFIGURATIONS

One of the most overlooked security threats is out-of-the-box settings or features designed to showcase cutting-edge functionality in an attempt to differentiate a given product from similar devices. Let us briefly look at some examples where default configurations landed the owners of consumer devices in hot water.

Owned Out of the Box

The Eee PC 701 (en.wikipedia.org/wiki/ASUS_Eee_PC) is a subnotebook class device shipped with a custom distribution of Linux. The custom configuration of Xandros included several services turned on by default to facilitate ease of use targeted at less technical end users. The Eee PC was exploitable out of the box to a standard Metasploit module. This allowed anyone who was able to connect to the Eee PC Samba service to acquire root on the box with almost no effort! Had Samba been turned off by default, or the default configuration changed to require the user to enable Samba, the vulnerability would have still existed, but at least the attack surface would've been greatly reduced until a patch could have been issued.

Standard Passwords

Every device that requires a user login comes with the chicken-and-egg problem of how to communicate the initial default device password to the user. Many devices have standard passwords or insecure security settings (to see some examples, Phenoelit maintains a Default Password List at phenoelit.org/dpl/dpl.html). The worst offenders

of this category are embedded routers that often share default passwords across entire product lines. The number of routers with remote administration and the default password still enabled on the Internet is staggering!

The problem is so prolific that it has enabled a new class of vulnerability chaining attacks for client exploitation. An attacker will use a cross-site response forgery to log in to the router and change the settings to redirect the users to a malicious DNS and other services.

Default passwords and configurations are not limited to routers and PCs. Another example is the recent rediscovery of the default password to Triton ATMs. Every Triton ATM shipped with the same administrative access code allowing anyone with the code to print a transaction log or perform other administrative functions on the ATM. In many cases, the transaction log revealed the account numbers and names of the customers who used the machine.

Bluetooth

The eternal wellspring of cell phone insecurity is Bluetooth (en.wikipedia.org/wiki/Bluetooth). Phones sync, make calls, transfer data, tether, and offer nearly every service over the Bluetooth protocol. Yet some phones are still shipped with discovery mode enabled by default, allowing any attacker to discover and connect with the device. Bluetooth has enabled attackers to penetrate networks, steal contacts, and social engineer individuals for nearly a decade.

One simple, inexpensive off-the-shelf tool to help with Bluetooth hardware hacking is Ubertooth (Figure 9-10), which can be found at ubertooth.sourceforge.net. Among other things, it allows for the sniffing and playback of Bluetooth frames across all 80 bluetooth channels in the 2.4 GHz ISM band and for a low cost of \$120 (see Figure 9-11). The hardware can be purchased from SparkFun (sparkfun.com).

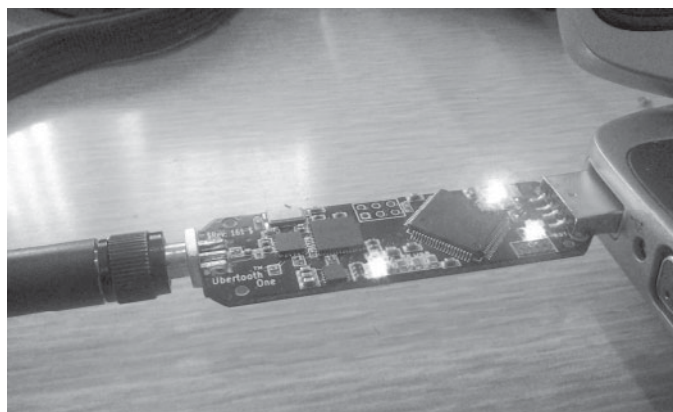
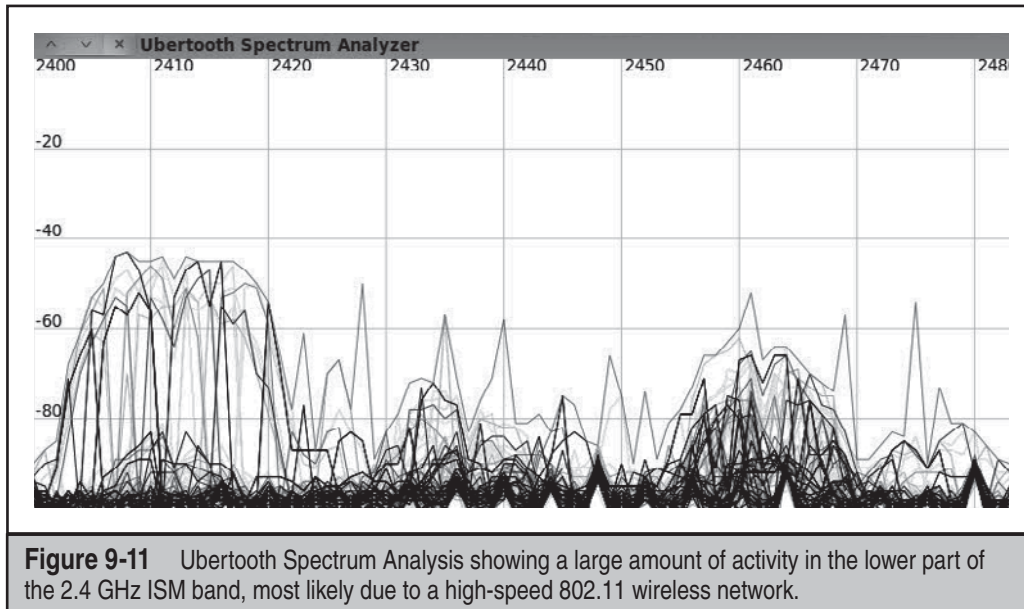


Figure 9-10 The Ubertooth One device



REVERSE ENGINEERING HARDWARE

To this point, we've discussed attacks against common off-the-shelf (COTS) devices like ATA disk drives and USB sticks. What do attackers do when confronted by more customized and complex devices? This section lays out various approaches to begin reverse engineering hardware devices to unlock the information inside.

Mapping the Device

Removing the cover of a device is the first step in reversing engineering hardware. The goal is to get access to the internal circuitry. The process is usually straightforward, likely just a few screws to remove. If the device is glued shut, a heatgun and a prying tool should easily give access. Some devices may be completely hermetically sealed, meaning the external device housing will need to be (gently) destroyed. Some devices may even employ special security screws; however, the bits to fit these can easily be found online. Many devices are built from COTS components that are often well documented in spec sheets on the manufacturer's website, which often provide descriptions of the functions, pinouts, and operating specifications.

Removing Physical Protections

There may be epoxy, conformal coating, or other physical protections on the PCB hiding integrated circuit (IC) chips of interest. Epoxy can be removed with a nitric acid process. It is strongly suggested that only those very familiar with the process of nitric acid

de-encapsulation and the safe handling procedures of nitric acid (HNO₃) use this method for gaining access to ICs protected with epoxy. Conformal coating can be removed with MG Chemicals 8310 Conformal Coating Stripper. A careful person may even be able to use a Dremel. Another mostly noninvasive way to look under these coatings would be with X-ray imaging.

Identifying Integrated Circuit Chips

Identifying integrated circuit, or IC, chips is an important part of the reverse engineering process. And it is often a fun and easy exercise in using a search engine and one of the first steps in understanding an embedded system. All ICs have datasheets that you can find by entering the part number into Google or one of the many online parts retailers such as Newark or DigiKey. Datasheets contain a wealth of information on parts packaging, electrical characteristics and maximum limits, pin diagrams, and some application notes and examples.

ICs come in a variety of packages, and although DIP chips are easy to work with, in a modern production embedded system, you will most likely come across surface mount chips. The top of an IC is typically marked with a dot or a notch, and the pins are numbered counterclockwise from that mark. Most ICs have an identifying code printed at the top, which is generally a model number along with possibly some packaging, temperature, and materials codes, and a serial number (see Figure 9-12). Smaller IC packages may use a compacted form of the model number, which may create some work in trying to identify the exact chip in question.

Larger ICs in DIP form factor can be removed easily with solder wick, and surface mount chips can be removed either with ChipQuik from chipquik.com or with a hot air station.

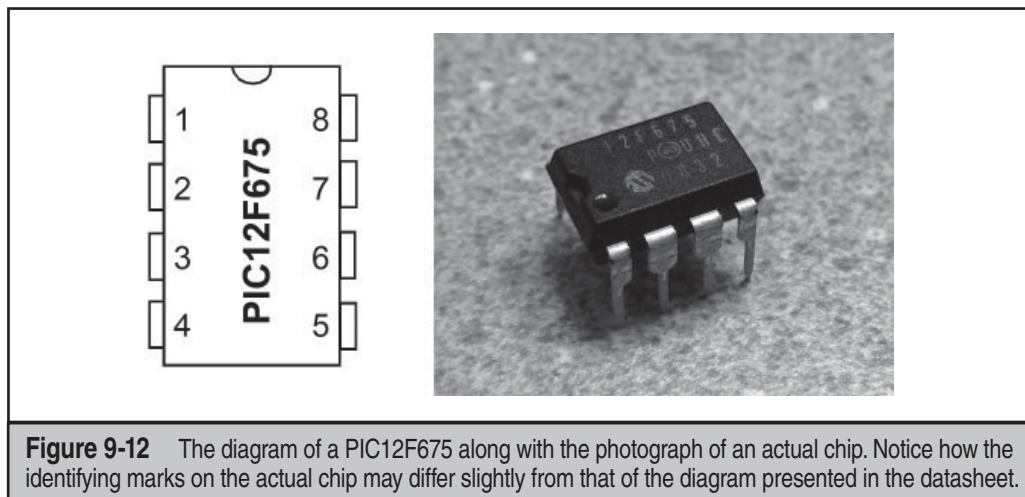


Figure 9-12 The diagram of a PIC12F675 along with the photograph of an actual chip. Notice how the identifying marks on the actual chip may differ slightly from that of the diagram presented in the datasheet.

Microcontrollers A microcontroller (MCU) is a small CPU or system on a single IC, containing a processor, a tiny amount of memory, and some nonvolatile memory, usually in the form of Flash. Microcontrollers are widely used in embedded applications. Getting at the programming code of a microcontroller is a very helpful task to perform when hacking a hardware device. Many are readable via an off-the-shelf EEPROM programmer with few protections.

EEPROM Electrically Erasable Programmable Read-Only Memory (EEPROM) is a type of nonvolatile memory used in electronics to store small amounts of data—oftentimes system firmware code for a microcontroller or CPU—that must be saved when the power is removed. EEPROM can typically be read with an off-the-shelf EEPROM programmer and generally does not have strong security in place.

FPGAs Once in a while you may come across an FPGA in an embedded system, possibly by Altera or Xilinx, two of the most popular vendors. An FPGA is a field-programmable gate array with an extremely flexible chip that can be used to implement a wide variety of logical operations and can be reconfigured a countless number of times. FPGAs contain reconfigurable logical blocks that can be wired together to perform complex functions, create blocks of memory, or create a simple logic gate.

An FPGA is programmed using a hardware description language (HDL); two common ones are VHDL and Verilog. As in the case of microcontrollers, robust development toolkits for FPGAs are often free for download from the vendors. And important to the core of FPGA development is both an HDL development environment and an HDL simulator, much like a microcontroller emulator. Even the basics of VHDL and Verilog programming are outside the scope of this book, but it's worth mentioning in case the adventurous reader is interested in learning more about these types of chips.

The job of debugging an FPGA can be difficult due to a lack of internal visibility. Large FPGAs can contain entire systems on a single chip, and the visibility problem becomes exponentially more of an issue. There are typically two methods for accessing an FPGA system: First, nodes can be routed out to pins in the FPGA design and those pins can be analyzed using a traditional external logic analyzer. Second, a logic analyzer or debugger can be built into the core of the FPGA design and routed out via JTAG. Because JTAG facilities are increasingly common on embedded systems to provide a standardized debugging interface, you may be lucky enough to find such an interface. Otherwise, the only route left is a long and arduous process of trial and error using a logic analyzer to identify and decode the hidden meanings of the FPGA's external pins.

External Interfaces

A device is usually connected to the world with some form of external interface. Common interfaces include those of standard peripherals, networking, serial, HDMI, USB, wireless, and even test points of a JTAG. Any of these interfaces may offer a possible attack vector or potentially leak information. Look for any interface that can be connected to; some test points may even be hidden under a panel or sticker.

Identifying Important Pins

Figure 9-13 shows a mock pinout of a microcontroller chip common to many devices. Notice the small notch in the top. This lines up with a notch in the physical chip and allows you to tell which pin aligns to pin 0 or pin 21. For square chips, a circle or triangle is used instead. From the pinout, we can see there are the PWR and GND lines associated with power and ground. The pins most likely to interest reverse engineers are the TX and RX lines, as these generally are associated with a serial bus. The other lines are DL (digital lines) and AD (analog to digital or analog lines). The digital and analog input and output lines are normally wired to other components or take input from other devices. This information will be useful in sniffing and capturing intercomponent interactions.

Modern circuit boards are multilayer, with a minimum of 4 to 64 layers of silicon and metal. This can make tracing leads from one component to another difficult by visual inspection alone. To create a full component and bus map, use a multimeter with a toning function, as shown in Figure 9-14.

The toning function works by sending power from one of the multimeter leads to the other. When a wire is connected on both ends of the multimeter, it will beep, flash, or alert the user that a connection has been made. This confirms that the two components are connected even though the path can't be seen. Using specification sheets and a multimeter, a reverse engineer can create a full picture of how the components on the device interface.

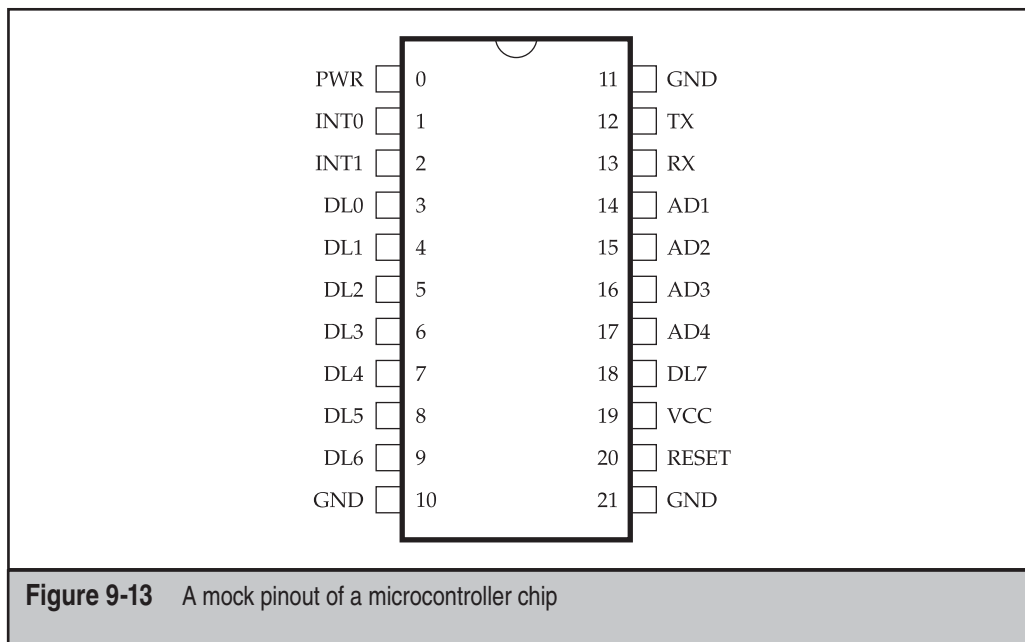


Figure 9-13 A mock pinout of a microcontroller chip

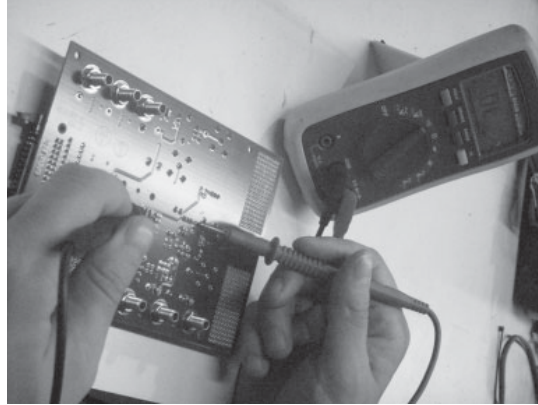


Figure 9-14 Using a multimeter to create a component and bus map

CAUTION

Some devices cannot handle the power supplied by a multimeter toning function. Applying too much power to the wrong components can damage or destroy the device; proceed at your own risk.

Sniffing Bus Data

Just like networks, buses on hardware transmit data from one component to another. In fact, a network could just be considered a multicomputer bus. The information going across a hardware bus is generally unprotected and thus susceptible to intercept, replay, and man-in-the-middle attacks. An exception to this rule is the information sent in DRM systems like HDMI-HSCP, which requires information be encrypted as it is sent from chip to chip.

Getting the information on the bus can be trivial or very difficult. Good reconnaissance helps identify which lines on the device are part of the bus you wish to intercept and what clock rate that information is traveling at. A logic analyzer like the one shown in Figure 9-15 allows you to see and record what signals are currently on the bus. These signals correspond to 1s or 0s denoting data that can be decoded later.

To perform a sniffing attack, attach the leads of the logic probe to the various chip or pin contacts as shown in Figure 9-16, and set the logic analyzer to receive signals as shown in Figure 9-17.

Larger leads may pose no real significant problem but logic probes may not be so easy to attach to. This situation may require a low-power stereo-microscope, a PCB trace repair kit, and some fine soldering work to bring out the appropriate contacts far enough



Figure 9-15 A logic analyzer views signals traversing a bus.

to get a good grasp on them, as shown in Figure 9-18. A good kit for this is the Thermo-Bond Cir-Kit by Pace, although keep in mind a complete starter kit may run approximately \$300.

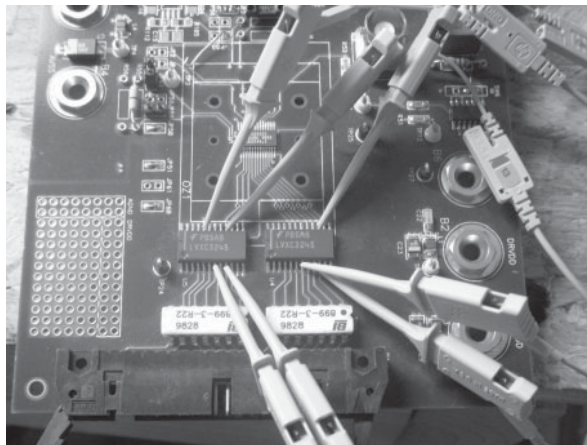


Figure 9-16 Attaching logic probes to various chips and pin contacts

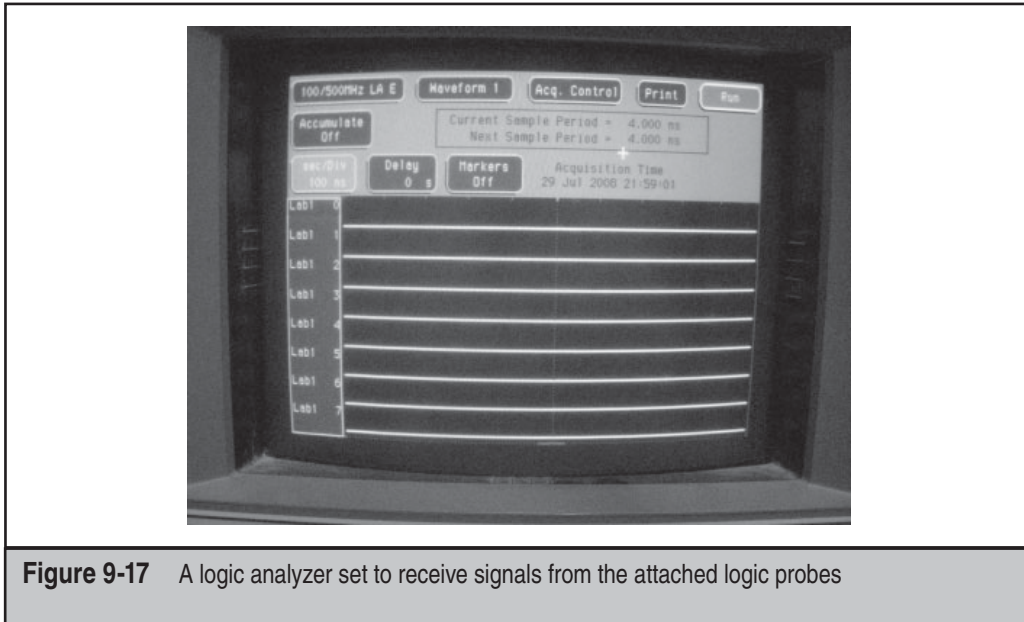


Figure 9-17 A logic analyzer set to receive signals from the attached logic probes

The data will appear in the logic analyzer in the raw, which isn't very user friendly. However, with a bit of work and some documentation from the chip maker, decoding the information is feasible. To make life easier, some logic analyzers have built-in decoders for common bus protocols like I2C, SPI, and Serial.

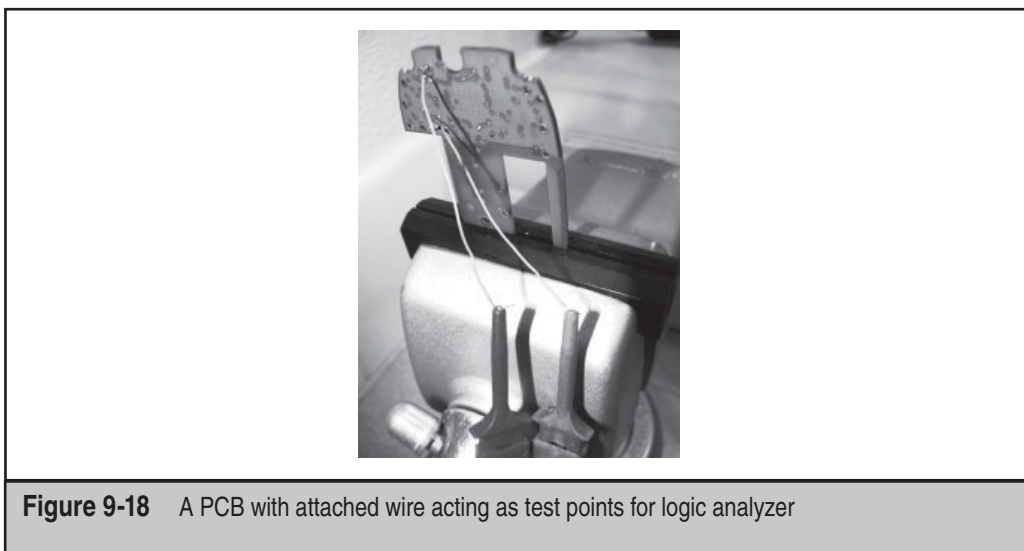


Figure 9-18 A PCB with attached wire acting as test points for logic analyzer

One can even send arbitrary and malformed signals to the pins to attempt to trigger some form of fault in much the same way application and protocol fuzzing is done. But this can have the added consequence of rendering the device damaged and useless.

Sniffing the Wireless Interface

Before the wireless interface can be accessed, a client device must be available, such as a basic transceiver, another wireless network card, or a Bluetooth device. Then layer 2 software attacks can be performed against the device, but if these aren't available to you, then you'll need to perform some reconnaissance. A first step in hacking the wireless interface of the device is to identify the device's FCC ID. The ID should be printed on the device, packaging, or in the manual. Every device that operates over radio frequency in the United States must be issued an FCC ID. The number is broken up into a three-character grantee code and a variable number of remanding characters. With this number, you can perform a search on the FCC website at fcc.gov/oet/ea/fccid/. This should give you the correct documents pertaining to the device. Some useful information should be found regarding the radio frequencies on which the device is to operate, as well as some internal diagrams.

By knowing the radio frequencies the device operates on, along with the type of modulation the device uses, *symbol decoding*, which is the lowest level of wireless decoding, should be possible. Symbol decoding is effectively decoding the lowest level bits from the wireless channel on which the device operates, similar to bus data from a physical bus line. A datasheet for one of the IC chips on the hardware device, the user manual, or FCC search site should confirm the RF frequencies used. With this information, you can perform the symbol decoding the help of a software-defined radio, of which you have a few choices such as WinRadio or USRP. Even with a software-defined radio, a significant amount of software programming may be required to get at the symbol stream from the wireless interface.

Firmware Reversing

Most embedded devices require some form of custom firmware to run. These firmware files are field upgradable and can be loaded by the user. Firmware upgrades are often hosted on manufacturers' websites or available upon request from the manufacturer. Looking inside of firmware files can lead to a plethora of juicy information about the device, such as default passwords, administrative ports, and debugging interfaces. The fastest way to inspect the firmware file is using a hex editor like 010 Editor, available from SweetScape Software. The 010 Editor is shown in Figure 9-19. Here the firmware image is loaded into the 010 Editor. From the decodes in the Editor, we can guess that AES encryption is being used.

Another common tool is IDA Pro, not only an absolute necessity in the world of software reverse engineering, but also indispensable when it comes to reverse engineering the firmware of any embedded device, as it supports over 50 families, which in total means hundreds of individual processors. Oftentimes, the firmware image is loaded directly by the microcontroller and execution starts at a fixed address, very much like an

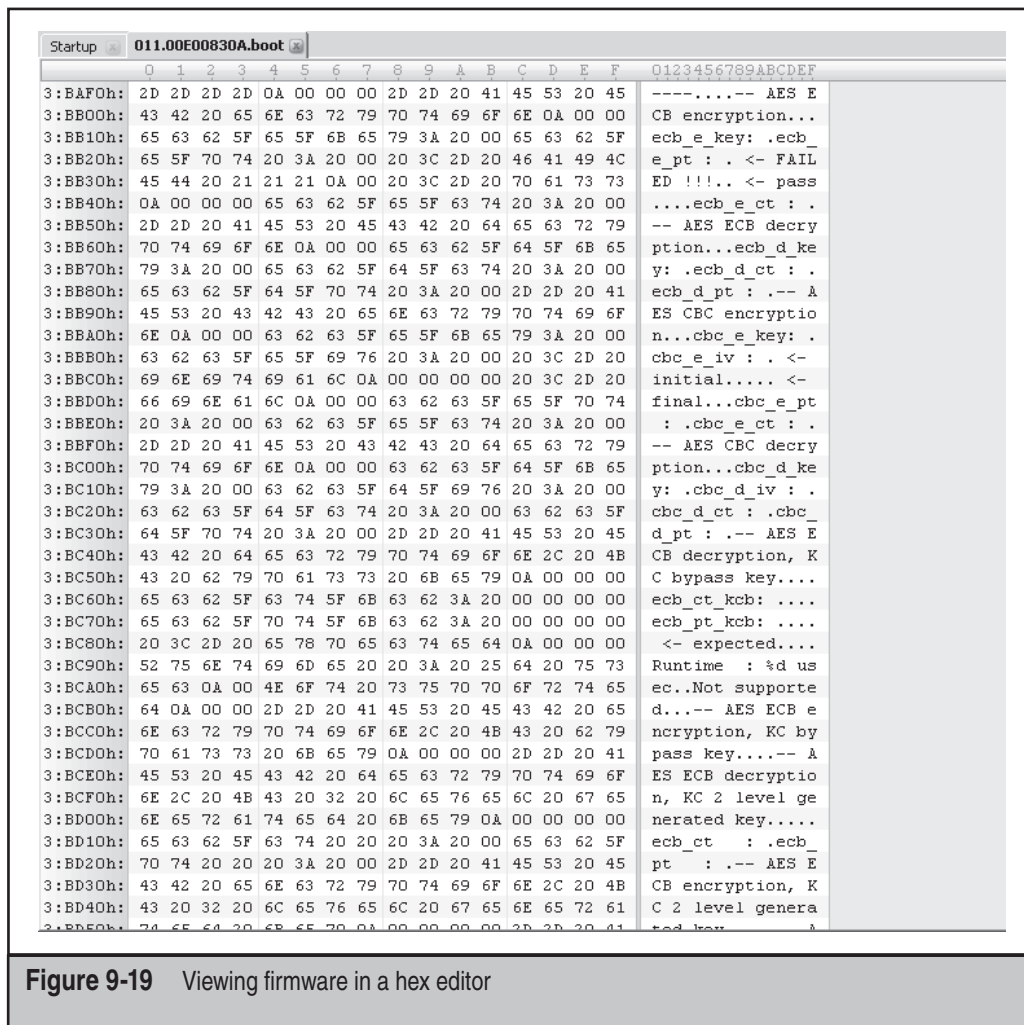


Figure 9-19 Viewing firmware in a hex editor

MS-DOS COM file. In IDA Pro, this translates into determining the entry point, something that can often be found with the assistance of the microcontroller datasheet.

Another useful tool when looking at custom firmware or binaries is the UNIX command strings. The strings utility prints all of the ASCII strings from a binary. Many developers hard-code passwords, keys, or other useful information for an attacker. Next, we've listed some example output from running strings against some firmware:

```
bootcmd=run setargs; run add${bootfs}; bootn
bootdelay=1
baudrate=115200
```



```
ethaddr=00:10:25:07:00:00
mtdids=nand0=Nand
mtdparts=mtdparts=Nand:2M(Boot),24M(FS1),24M(FS2),14M(RW)
addcramfs=setenv bootargs ${bootargs} root=/dev/mtdblock_robbs1 ro
addnfs=setenv bootargs ${bootargs}
ip=${ipaddr}:${serverip}:::${ethport} root=/dev/nfs rw
nfsroot=${serverip}:${rootpath},tcp,nfsvers=3
setargs=setenv bootargs console=ttyS0,0
autostart=yes
ethport=eth0
rootpath=/rootfs
ipaddr=192.168.0.2
serverip=192.168.0.1
bootfs=cramfs
bootcmd=boota
```

From the output, we can see that the file system used is cramfs. We will use this information to explore more of the firmware. Let's try and mount the firmware image using the Linux/UNIX mount command:

```
adam@blackbox:/tmp$ sudo mount -o loop -t cramfs
/home/adam/OAA.EAAAA /tmp/cram/
adam@blackbox:/tmp$ cd /tmp/cram
adam@blackbox:/tmp/cram$ ls -al
total 14
drwxrwxrwx 1 7423 178 1476 1969-12-31 16:00 bin
drwxrwxrwx 1 7423 178 284 1969-12-31 16:00 dev
drwxrwxrwx 1 7423 178 584 1969-12-31 16:00 etc

drwxrwxrwx 1 7423 178 16 1969-12-31 16:00 home
drwxrwxrwx 1 7423 178 0 1969-12-31 16:00 images
drwxrwxrwx 1 7423 178 1720 1969-12-31 16:00 lib
drwxrwxrwx 1 7423 178 0 1969-12-31 16:00 media
drwxrwxrwx 1 7423 178 0 1969-12-31 16:00 mnt
drwxrwxrwx 1 7423 178 0 1969-12-31 16:00 nvram
drwx----- 1 7423 178 16 1969-12-31 16:00 opt
drwxrwxrwx 1 7423 178 0 1969-12-31 16:00 proc
drwxrwxrwx 1 7423 178 0 1969-12-31 16:00 pvr
drwxrwxrwx 1 7423 178 640 1969-12-31 16:00 sbin
drwxrwxrwx 1 7423 178 0 1969-12-31 16:00 sys
drwxrwxrwx 1 7423 178 0 1969-12-31 16:00 tmp
drwxrwxrwx 1 7423 178 84 1969-12-31 16:00 usr
drwxrwxrwx 1 7423 178 124 1969-12-31 16:00 var
adam@blackbox:/tmp/cram$
```

Easy as could be! Luckily for us, this firmware image didn't include any custom protections such as packing, encoding, or encryption, which can range from trivial to incredibly difficult to defeat. From here, we are free to explore more of the custom Linux distribution that is included on the device and probe for holes or other weaknesses in the exposed binaries and services.

In this case, the easiest approach is to navigate around the file system looking for sensitive files, such as the public and private keys used in authentication. The UNIX `find` command helps us locate relevant items. Let's look for a few common key names.

```
adam@blackbox:~# find /tmp/cram -name *key
adam@blackbox:~# find /tmp/cram -name *cert
adam@blackbox:~# find /tmp/cram -name *pgp
adam@blackbox:~# find /tmp/cram -name *gpg
adam@blackbox:~# find /tmp/cram -name *der
adam@blackbox:~# find /tmp/cram -name *pem
/tmp/cram/etc/certs/ca.pem
/tmp/cram/etc/certs/clientca.pem
/tmp/cram/etc/certs/priv.pem
```

Bingo! Now that we have the public and private key files, we can forge an SSL connection and act like a trusted device on the private network.

Another attack vector, present more often than would be expected, is the (hopefully) unintentional backdoor in the form of testing code that was not removed after the development and testing process. Some of the places these may be found include hidden physical interfaces, architecture-specific debugging interfaces, diagnostic and serial ports, and defunct development code. Some examples of these in the wild are Intel's NetStructure cryptographic accelerator administrator access, Palm OS Debug Mode, or the Sega Dreamcast mask-ROM BIOS standard CD-ROM booter.

When reverse engineering the code from an IDA or whatever assembly-level debugging tool is available for the platform, a hacker should be on the lookout for code that apparently bypasses security measures by hardcoded authentication data or a special sequence of input. Here is one such backdoor found in the wireless serial number authentication code of a medical device. As you can see, after the normal serial number checks occur, a second serial number check occurs for the serial number 0x12 0x34 0x56:

```
ROM:00834694 serial_incorrect:
ROM:00834694
ROM:00834694     mov.b    #5, r6l
ROM:00834696     mov.b    r6l, @word_A06FA6+1:32
ROM:0083469C     mov.b    @serialbyte1:32, r6l ; RF serial byte 1
ROM:008346A2     cmp.b    #0x12, r6l          ; is 0x12?
ROM:008346A4     bne      loc_8346BA:8
ROM:008346A6     mov.b    @serialbyte2:32, r6l ; RF serial byte 2
ROM:008346AC     cmp.b    #0x34, r6l          ; is 0x34?
ROM:008346AE     bne      loc_8346BA:8
```

```
ROM:008346B0    mov.b    @serialbyte3:32, r6l    ; RF serial byte 3
ROM:008346B6    cmp.b    #0x56, r6l                ; is 0x56?
ROM:008346B8    beq      loc_8346D0:8
```

Now that the attacker has uncovered a backdoor, any client can be programmed with the special 0x12 0x34 0x56 serial number and gain full control over the medical device, completely bypassing the security mechanism.

EEPROM Programmers

The easiest way to typically get at the firmware of many chips is simply a universal EEPROM programmer. Numerous manufacturers and models are available for a wide variety of budgets, anywhere from \$200 for an inexpensive PICSTART plus or ChipMax, to the very robust \$1200 B&K Precision 866B (shown in Figure 9-20), and well beyond. Typically, after the correct IC chip is identified, which is usually a microcontroller, microprocessor, or some form of external EEPROM chip, the chip is inserted into the EEPROM reader socket, and then it's simply a matter of running the `read` command from within the EEPROM reader software application. Often the chip's production packaging is surface mounted (it would make things easier if the chip were left on the PCB). If this is the case, either some type of surface mount adapter can be used, or the programmer may provide an in-circuit serial programming (ICSP) interface, so the chip can be directly jumped "in-circuit" on board. Numerous configurations for adapters and ICSP connectors can be found online.



Figure 9-20 A B&K Precision 866B EEPROM programmer with microcontroller inserted for programming

After the firmware image is read back, there are a number of possibilities. Either hex-edit the firmware from within the EEPROM reader application, or save out an Intel HEX file and use it in other applications as a number of development tools support this format. The Intel HEX file format is used for storing binary information, for instance for programming microcontrollers and EPROMs, and has been in use since the 1970s.

Given a HEX file, writing back firmware to a chip is just as easy. It's just a matter of loading it into the EEPROM software program and then running the `write` command.

Some chips have read and write security, which allow for either blocking reads from firmware until an erase operation is first performed or disallowing any subsequent writes after the first one. It is best to check the datasheet for any security mechanisms such as flash read protection. Generally, the average reverse engineer would not have much in the way of tools to circumvent this type of protection, but it is possible that, with the help of more advanced and very expensive tools such as FIBs, micro-positioners, and tunneling microscopes, it would be possible to circumvent it; however, this is well beyond the scope of this book.

Microcontroller Development Tools

All microcontrollers have some sort of development tool. Often the chip manufacturers provide these tools for free download. Alternatively, a number of free tool chains are available for Linux, many of which are included in the major distribution package managers. Many HEX files can be loaded directly into the appropriate development tool to be analyzed, disassembled, debugged, and emulated.

One such toolkit is MPLAB IDE for the Microchip PIC series of microcontrollers. MPLAB IDE is a fully integrated development environment for the PIC microcontrollers, with a complete software emulator, line debugger, assembler, and optional free C compiler. It also integrates with various hardware devices. Like most of the toolkits, MPLAB includes many tutorials for the beginning user. It appears that most chip vendors want to do whatever they can to get their chips out there and are, therefore, willing to provide support and free tools to facilitate this. It is in your best interest to peruse the vendor websites after identifying the main control chips on an embedded device.

ICE Tools

An in-circuit emulator (ICE) is a device to assist with the debugging of a hardware device in-circuit or while the device is in operation. In many ways, this term overlaps with JTAG (covered next), and ICE tools provide many of the same features JTAG provides if a hardware device supports it. The term *emulator* is somewhat of a misnomer as hardware is rarely emulated anymore. Instead ICE performs the work of a debugger by providing a window into the operation of the hardware.

In-circuit emulators are essential for any serious debugging operation since many hardware systems lack the IO niceties of typical computers such as keyboards and screens. These in-circuit emulators provide a window into the inner workings of the hardware device, with all of the power of your computer to help solve any debugging

problems. In fact, without some form of ICE, even debugging the simplest hardware issue can be an extremely difficult undertaking.

Unfortunately, there are as many ICE tools as there are chips that could use them, so it depends on the specific application you are looking to debug to determine the correct ICE tool to use. Some common ones are the MPLAB ICE tools for the microcode PIC series of microcontrollers or AVR JTAGICE. The best thing to do, after identifying the correct controller chip on the hardware platform you are trying to debug, is to contact either the manufacturer or a site like Newark.com to see which ICE solutions you have available.

JTAG

The most common ICE type of interface found on modern embedded systems is the JTAG interface. Joint Test Action Group, or JTAG (see en.wikipedia.org/wiki/JTAG), is a testing interface for printed circuit boards and other integrated circuits (ICs). JTAG was designed to test if the interfaces between components on a board were properly assembled post-manufacturing. Thus it allows an attacker to send and receive signals to each IC or component on the board. This makes JTAG a great resource to debug an embedded system or device when simple reversing doesn't yield results. Figure 9-21 shows a USB-to-JTAG device cable that allows easy interface from PCs to devices for purposes of hardware-level debugging.

Unfortunately, with JTAG, one size or shape does not fit all. The JTAG interfaces for several common embedded processors (ARM, Altera, MIPS, Atmel) all come in different pin counts ranging from 8 to 20 and configurations that are single row, dual row, and so

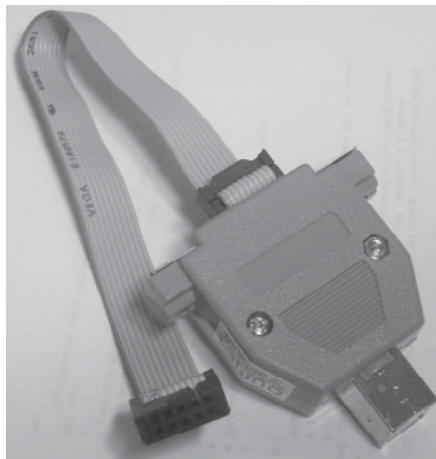


Figure 9-21 A USB-to-JTAG cable

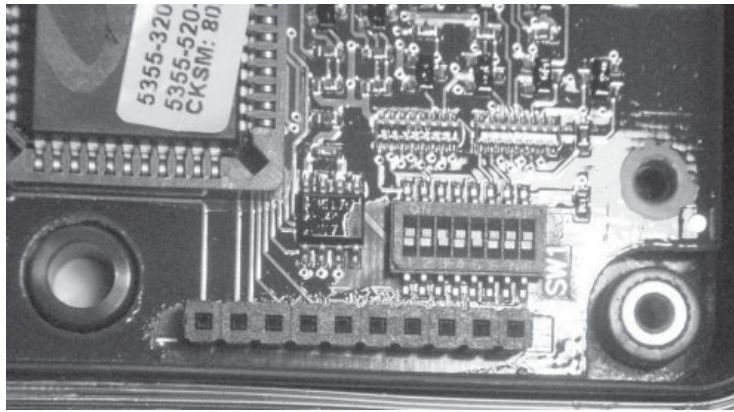


Figure 9-22 A custom JTAG interface

on. This can mean finding, buying, or building a new JTAG-to-PC cable for each device to be reversed. The software interface used will depend on which processor or device is being debugged. Luckily, most vendors supply debugging tools directly with their IDE or other interface. Figure 9-22 shows a custom JTAG interface on a device and Figure 9-23 shows a JTAG “wiggler” connected to a device.

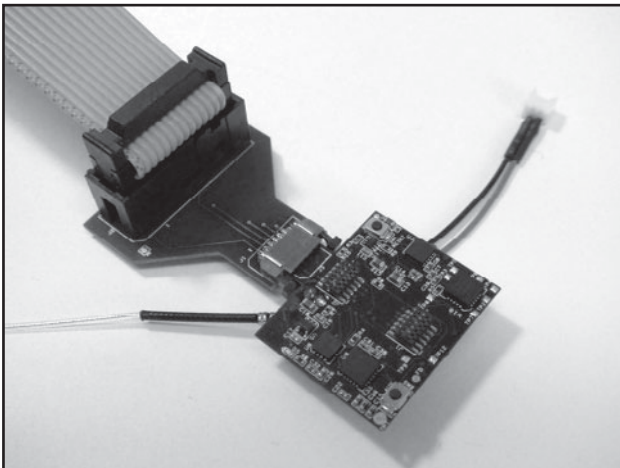


Figure 9-23 An inexpensive JTAG “wiggler” connected to a device for debugging

Barring access to vendor tools, there are several open projects that provide tools to interface with JTAG for ARM-based processors. The easiest to use are available from the OpenOCD project, which provides binaries for Windows and integration into the Eclipse development environment. They can be acquired at yagarto.de.

A larger more ambitious project is the UrJTAG project, which supports a wide range of JTAG interfaces and devices. The UrJTAG tools are available from urjtag.org.

SUMMARY

Despite the ongoing transition to digital formats, information is still held behind traditional locks and in hardware devices that are the ultimate protector of its confidentiality, integrity, and availability. We hope this chapter has prompted you to reconsider your overall program of protection for digital information and to include threats from physical attacks as well as the many logical threats catalogued in this book.